

PHÁT TRIỂN ỨNG DỤNG JAVA

Bài tập lớn Lập trình hướng đối tượng – Thực hiện & Thuyết trình theo nhóm

Eclipse IDE | Swing GUI | JFreeChart | 10 nhóm – 10 đề tài

Thời lượng: 2 giờ lập trình trên lớp + 1 tuần hoàn thiện + 15 phút thuyết trình & phản biện / nhóm

Mục tiêu học tập

- Vận dụng 4 nguyên lý OOP: Đóng gói, Kế thừa, Đa hình, Trừu tượng — và **giải thích được vì sao** dùng chúng
- Thiết kế phân cấp lớp có ý nghĩa nghiệp vụ, biểu diễn bằng sơ đồ lớp UML
- Xây dựng ứng dụng Swing nhiều cửa sổ: JFrame, JDialog, menu, toolbar, status bar
- Xử lý sự kiện, layout manager, JTable/JList, vẽ đồ họa với Graphics2D
- Tích hợp JFreeChart để trực quan hóa dữ liệu
- Làm việc nhóm, quản lý mã nguồn bằng Git, viết báo cáo kỹ thuật và thuyết trình – phản biện

Thông tin tổ chức

- Số nhóm: **10** (4–5 sinh viên/nhóm)
- Mỗi nhóm **1 đề tài riêng** (bốc thăm)
- Lập trình trên lớp: 2 giờ
- Hoàn thiện + báo cáo: 1 tuần
- Thuyết trình: 10 phút + phản biện 5 phút
- IDE: Eclipse, JDK 17 (tối thiểu 11)
- Thư viện: Swing, JFreeChart 1.5.x
- Quản lý mã nguồn: Git (GitHub/GitLab)

Danh sách 10 đề tài

Nhóm	Đề tài	Tên tiếng Anh	Trọng tâm kỹ thuật
1	**Quản lý Thư viện**	Library Manager	Quản lý tài liệu (sách, tạp chí, luận văn), mượn – trả và thống kê lượt mượn
2	**Quản lý Nhân sự & Bảng lương**	HR Payroll System	Quản lý nhân viên nhiều loại hợp đồng, tính lương – thuế và phân tích quỹ lương
3	**Ngân hàng Mini**	Mini Banking	Quản lý tài khoản, giao dịch nạp – rút – chuyển khoản và biến động số dư
4	**Đặt phòng Khách sạn**	Hotel Booking	Sơ đồ phòng trực quan, đặt phòng theo ngày, kiểm tra trùng lịch và lập hóa đơn
5	**Quầy bán hàng Quán cà phê**	Coffee Shop POS	Gọi món, áp dụng khuyến mãi, thanh toán và thống kê doanh thu
6	**Đăng ký Học phần**	Course Registration	Sinh viên đăng ký lớp học phần, kiểm tra ràng buộc và xem thời khóa biểu tuần
7	**Trình soạn thảo Văn bản Mini**	Mini Text Editor	Soạn thảo nhiều tab, mở/lưu file, tìm – thay thế, hoàn tác và thống kê văn bản
8	**Mô phỏng Nút giao thông**	Traffic Intersection Simulator	Mô phỏng hoạt hình ngã tư có đèn tín hiệu và nhiều loại phương tiện

Nhóm	Đề tài	Tên tiếng Anh	Trọng tâm kỹ thuật
9	**Giám sát Ao nuôi tôm IoT**	Smart Aquaculture Monitor	Dashboard giám sát cảm biến chất lượng nước theo thời gian thực và cảnh báo vượt ngưỡng
10	**Trò chơi Rắn săn mồi**	Snake Game	Game 2D có nhiều màn chơi, nhiều loại mồi và bảng xếp hạng điểm

Ghi chú: mức độ khó của các đề là tương đương. Đề 1–6 thiên về nghiệp vụ & dữ liệu (CRUD, ràng buộc, exception); đề 7–10 thiên về sự kiện, đồ họa và thời gian thực. Tất cả đều phải đạt đủ yêu cầu chung ở mục 3 và 4.

1. Tổ chức & tiến độ thực hiện

Bài tập được thực hiện qua 4 giai đoạn. Ngày cụ thể do giảng viên thông báo; nhóm ghi lại tiến độ thực tế vào báo cáo.

Giai đoạn	Thời điểm	Công việc	Kết quả phải có
0. Giao đề	Buổi giao đề (cuối giờ)	Lập nhóm, bầu nhóm trưởng, bốc thăm đề; đọc kỹ thẻ đề; tạo repository Git và mời giảng viên vào repo.	Danh sách nhóm + link repo gửi GV
1. Thiết kế	Trước buổi lập trình	Vẽ sơ đồ lớp UML phác thảo (giấy hoặc draw.io/PlantUML); phân công ai viết lớp nào; chuẩn bị icon PNG 16×16, 32×32.	Ảnh sơ đồ lớp v1 + bảng phân công
2. Lập trình trên lớp	2 giờ trên lớp	Code theo phân bổ thời gian trong thẻ đề. Mục tiêu cuối buổi: chạy được MainWindow + model + ít nhất 1 JDialog + 1 biểu đồ (dữ liệu giả cũng được).	Commit "end of lab session" trên Git
3. Hoàn thiện	1 tuần sau buổi lập trình	Hoàn thiện đủ 8 bước, xử lý lỗi, kiểm thử, làm BONUS (nếu có), viết báo cáo, làm slide, tập dượt thuyết trình.	Sản phẩm nộp đủ theo mục 8.3
4. Thuyết trình	Buổi bảo vệ	10 phút trình bày + 5 phút phản biện. Giảng viên hỏi **đích danh** từng thành viên.	Điểm nhóm + điểm phản biện cá nhân

Lịch thuyết trình gợi ý (2 ca)

Ca	Nhóm	Thời lượng	Ghi chú
Ca 1	Nhóm 1 → 5	5 × 15′ = 75 phút	Chuẩn bị máy chiếu, chạy thử trước 10 phút
Ca 2	Nhóm 6 → 10	5 × 15′ = 75 phút	Nghỉ 10 phút giữa 2 ca

Thứ tự thuyết trình trong mỗi ca có thể được bốc thăm lại ngay đầu buổi — mọi nhóm phải sẵn sàng từ đầu ca.

Phân công vai trò trong nhóm (gợi ý)

Mỗi thành viên đều phải **viết code** (ít nhất 2 lớp có nghĩa) và **nói** trong buổi thuyết trình. Vai trò dưới đây chỉ là trách nhiệm chính, không phải giới hạn công việc.

Vai trò	Trách nhiệm chính	Lớp/thành phần thường phụ trách
Nhóm trưởng / Tích hợp	Chia việc, quản lý Git (merge, giải quyết xung đột), đảm bảo tiến độ, ghép các phần lại chạy được.	`Main`, `MainWindow`, cấu trúc package
Model & Nghiệp vụ	Thiết kế phân cấp lớp, lớp trừu tượng, interface, exception, quy tắc nghiệp vụ.	Các lớp trong `model/`, `service/`, `repository/`
Giao diện (GUI)	Layout, các JDialog, menu/toolbar, xử lý sự kiện, kiểm tra dữ liệu nhập.	`ui/`, `ui/dialogs/`, TableModel, Renderer
Dữ liệu & Biểu đồ	Sinh dữ liệu mẫu/mô phỏng, JFreeChart, đọc ghi file, Timer.	`util/ChartHelper`, `ChartWindow`, I/O

Vai trò	Trách nhiệm chính	Lớp/thành phần thường phụ trách
Kiểm thử & Báo cáo	Viết kịch bản kiểm thử, ghi lỗi, tổng hợp báo cáo, chuẩn bị slide và kịch bản demo.	Test case, báo cáo, slide (vẫn phải code ≥ 2 lớp)

Nhóm 4 người: gộp vai trò "Kiểm thử & Báo cáo" vào các vai trò còn lại.

2. Chuẩn bị môi trường

Thư viện cần có

- JDK 17 (tối thiểu Java SE 11) — Swing có sẵn
- JFreeChart 1.5.x (`jfreechart-1.5.x.jar`)
- Lưu ý: JFreeChart **1.5.x không cần JCommon** (JCommon chỉ dùng cho bản 1.0.x cũ)
- Tùy chọn: FlatLaf (giao diện hiện đại)

Thêm JAR trong Eclipse

- Chuột phải project › Properties
- Java Build Path › tab Libraries
- Chọn Classpath › Add External JARs…
- Chọn file JAR › Apply and Close
- Hoặc dùng Maven (khuyến khích) — xem bên dưới

Khai báo Maven

```
<!-- pom.xml (nếu dùng Maven Project trong Eclipse) -->
<dependency>
    <groupId>org.jfree</groupId>
    <artifactId>jfreechart</artifactId>
    <version>1.5.4</version>
</dependency>
```

Cấu trúc project bắt buộc

Tên project: `OOP_NhomXX_TenDeTai` (ví dụ `OOP_Nhom03_MiniBanking`). Package gốc: `vn.edu.ptit.oop.nhomXX`.

```
src/
  vn/edu/ptit/oop/nhomXX/
    Main.java                   // Điểm vào – dùng SwingUtilities.invokeLater
    common/BaseWindow.java      // Lớp trừu tượng BẮT BUỘC (mục 5)
    model/                      // Lớp nghiệp vụ: lớp trừu tượng, lớp con, enum
    service/                    // Xử lý nghiệp vụ (tính toán, kiểm tra ràng buộc)
    repository/                 // Interface + cài đặt lưu trữ dữ liệu
    exception/                  // Exception tự định nghĩa
    ui/MainWindow.java          // extends BaseWindow
    ui/panels/                  // Các JPanel tự vẽ / panel con
    ui/dialogs/                 // Các JDialog (Form, About, Settings…)
    util/ChartHelper.java       // Tạo biểu đồ JFreeChart
  resources/icons/              // app.png, add.png, edit.png… (16x16, 32x32)
docs/                           // Sơ đồ UML, ảnh chụp màn hình
README.md                       // Cách chạy, thành viên, đề tài
```

3. Yêu cầu OOP chung (mọi nhóm)

Mỗi nguyên lý phải có **ít nhất một ví dụ có ý nghĩa nghiệp vụ**, không chấp nhận lớp "cho có" (ví dụ lớp trừu tượng không có phương thức trừu tượng nào, hoặc getter/setter chỉ gán mà không có lý do).

Khái niệm	Cách thể hiện trong ứng dụng Swing	Trọng số
Đóng gói (Encapsulation)	Trường `private`; getter/setter có kiểm tra hợp lệ; không để lộ collection nội bộ (trả về bản sao hoặc `Collections.unmodifiableList`).	20%

Khái niệm	Cách thể hiện trong ứng dụng Swing	Trọng số
Kế thừa (Inheritance)	Phân cấp lớp nghiệp vụ (≥ 3 lớp con) **và** kế thừa Swing: `extends BaseWindow/JPanel/JDialog/AbstractTableModel`.	20%
Đa hình (Polymorphism)	Override phương thức nghiệp vụ được gọi qua tham chiếu kiểu cha (`List<LớpCha>`); override `paintComponent()`, `actionPerformed()`… **Hạn chế dùng `instanceof`/`switch` theo kiểu.**	20%
Trừu tượng (Abstraction)	Lớp trừu tượng nghiệp vụ + `BaseWindow.initUI()`; ít nhất **1 interface** do nhóm tự định nghĩa.	20%
Kết hợp (Composition)	MainWindow chứa các panel, toolbar, menu, chart; đối tượng nghiệp vụ chứa đối tượng khác (Order chứa OrderItem…).	10%
Xử lý sự kiện & ngoại lệ	ActionListener, MouseListener, WindowListener…; ít nhất 1 exception tự định nghĩa; không có khối `catch` rỗng.	10%

4. Thành phần giao diện bắt buộc

Kiến trúc cửa sổ

- MainWindow (extends BaseWindow) dùng BorderLayout
- Ít nhất **2 JDialog** (1 form nhập liệu + About)
- Menu bar: File, View, Help (có mnemonic, phím tắt)
- Toolbar có nút icon PNG 16×16 + tooltip
- Status bar ở đáy, cập nhật theo thao tác
- Icon trên thanh tiêu đề (`setIconImage`)

Dữ liệu & trực quan hóa

- Ít nhất **2 biểu đồ** JFreeChart thuộc 2 loại khác nhau, có tiêu đề và nhãn trục
- JTable hoặc JList hiển thị dữ liệu, bọc trong JScrollPane
- Dữ liệu thay đổi động (Timer hoặc thao tác người dùng) và biểu đồ cập nhật theo
- Kiểm tra dữ liệu nhập, thông báo lỗi thân thiện
- Giao diện tiếng Việt có dấu hiển thị đúng
- Look & Feel hoặc bảng màu thống nhất

5. Mã khởi đầu bắt buộc: BaseWindow

Tất cả các nhóm **phải** dùng lớp trừu tượng dưới đây làm lớp cha của `MainWindow`. Được phép **bổ sung** phương thức `protected`, nhưng không được bỏ phương thức trừu tượng `initUI()`.

```
package vn.edu.ptit.oop.nhomXX.common;

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.net.URL;

public abstract class BaseWindow extends JFrame {
    protected JMenuBar menuBar;
    protected JMenu    fileMenu, viewMenu, helpMenu;
    protected JToolBar toolBar;
    protected JPanel   statusBar;
    protected JLabel   statusLabel;

    public BaseWindow(String title, int width, int height) {
        super(title);
        setSize(width, height);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());
        loadIcon();          // icon thanh tiêu đề
        initMenuBar();       // File / View / Help
        initToolBar();       // thanh công cụ
        initStatusBar();     // thanh trạng thái
        initUI();            // <<< TRỪU TƯỢNG: mỗi nhóm tự cài đặt
    }

    /** Lớp con xây dựng vùng nội dung chính và thêm vào BorderLayout.CENTER. */
    protected abstract void initUI();

    /** Hook: lớp con override để hiển thị AboutDialog riêng của nhóm. */
    protected void showAbout() {
        JOptionPane.showMessageDialog(this, getTitle(), "About", JOptionPane.INFORMATION_MESSAGE);
    }

    protected void setStatus(String msg) { statusLabel.setText(" " + msg); }

    /** Nạp icon từ resources; trả về null thay vì ném NullPointerException. */
    protected static ImageIcon loadImageIcon(String path) {
        URL url = BaseWindow.class.getResource(path);
        return (url != null) ? new ImageIcon(url) : null;
    }

    /** Tiện ích thêm nút có icon vào toolbar. */
    protected JButton addToolButton(String iconPath, String tooltip, ActionListener action) {
        ImageIcon icon = loadImageIcon(iconPath);
        JButton btn = (icon != null) ? new JButton(icon) : new JButton(tooltip);
        btn.setToolTipText(tooltip);
        btn.setFocusable(false);
        btn.addActionListener(action);
        toolBar.add(btn);
        return btn;
    }

    private void loadIcon() {
        ImageIcon icon = loadImageIcon("/resources/icons/app.png");
        if (icon != null) setIconImage(icon.getImage());
    }

    private void initMenuBar() {
        menuBar  = new JMenuBar();
        fileMenu = new JMenu("File"); fileMenu.setMnemonic(KeyEvent.VK_F);
```

```
        viewMenu = new JMenu("View"); viewMenu.setMnemonic(KeyEvent.VK_V);
        helpMenu = new JMenu("Help"); helpMenu.setMnemonic(KeyEvent.VK_H);

        JMenuItem exitItem = new JMenuItem("Exit");
        exitItem.setAccelerator(KeyStroke.getKeyStroke(KeyEvent.VK_Q, InputEvent.CTRL_DOWN_MASK));
        // Gửi sự kiện đóng cửa sổ để WindowListener của lớp con (nếu có) được gọi
        exitItem.addActionListener(e ->
            dispatchEvent(new WindowEvent(this, WindowEvent.WINDOW_CLOSING)));
        fileMenu.add(exitItem);

        JMenuItem aboutItem = new JMenuItem("About...");
        aboutItem.addActionListener(e -> showAbout());   // gọi đa hình
        helpMenu.add(aboutItem);

        menuBar.add(fileMenu); menuBar.add(viewMenu); menuBar.add(helpMenu);
        setJMenuBar(menuBar);
    }

    private void initToolBar() {
        toolBar = new JToolBar();
        toolBar.setFloatable(false);
        add(toolBar, BorderLayout.NORTH);
    }

    private void initStatusBar() {
        statusBar = new JPanel(new BorderLayout());
        statusBar.setBorder(BorderFactory.createMatteBorder(1, 0, 0, 0, Color.LIGHT_GRAY));
        statusLabel = new JLabel(" Sẵn sàng");
        statusBar.add(statusLabel, BorderLayout.WEST);
        add(statusBar, BorderLayout.SOUTH);
    }
}
```

Main và khung MainWindow

```
public class Main {
    public static void main(String[] args) {
        // Mọi thao tác Swing phải chạy trên Event Dispatch Thread (EDT)
        SwingUtilities.invokeLater(() -> new MainWindow().setVisible(true));
    }
}

public class MainWindow extends BaseWindow {
    private LibraryRepository repo;      // KHÔNG viết "= new ..." ở đây (xem lưu ý bên dưới)
    private JTable table;

    public MainWindow() {
        super("Nhóm 01 – Quản lý Thư viện", 1100, 700);
    }

    @Override
    protected void initUI() {
        repo = new InMemoryLibraryRepository();      // khởi tạo trong initUI()
        addToolButton("/resources/icons/add.png", "Thêm tài liệu", e -> openAddDialog());
        table = new JTable(new ItemTableModel(repo));
        add(new JScrollPane(table), BorderLayout.CENTER);
        setStatus("Đã nạp " + repo.findAll().size() + " tài liệu");
    }

    @Override
    protected void showAbout() { new AboutDialog(this).setVisible(true); }
}
```

⚠ Lưu ý kỹ thuật quan trọng (sẽ được hỏi khi phản biện)

- **Constructor của BaseWindow gọi `initUI()`** — lúc đó constructor của MainWindow **chưa chạy xong**, các trường có khởi tạo tại chỗ (`private X x = new X();`) sẽ được gán **sau** khi `initUI()` kết thúc. Hệ quả: `NullPointerException` hoặc giá trị bị ghi đè. Cách làm đúng: khởi tạo các trường **bên trong `initUI()`** và không gán giá trị tại chỗ khai báo.
- Cửa sổ con (`ChartWindow extends JFrame`) phải dùng `DISPOSE_ON_CLOSE`, nếu dùng `EXIT_ON_CLOSE` thì đóng biểu đồ sẽ tắt luôn chương trình.
- Không chạy tác vụ lâu (đọc file lớn, `Thread.sleep`) trong `actionPerformed` — giao diện sẽ bị "đơ". Dùng `SwingWorker` hoặc `javax.swing.Timer`.
- Đặt font hỗ trợ tiếng Việt cho biểu đồ (xem đoạn mã 10.7) để tránh lỗi hiển thị ô vuông.

6. Đề bài của 10 nhóm

Mỗi nhóm nhận **một** thẻ đề dưới đây. Bước 1–7 là **bắt buộc**, bước 8 là BONUS (cộng tối đa 5 điểm). Nhóm được phép mở rộng thêm nhưng không được bỏ bớt yêu cầu. Sơ đồ lớp trong thẻ chỉ là **gợi ý tối thiểu**: nhóm có thể thiết kế khác nếu giải thích được lý do.

Ký hiệu sơ đồ: `A ◄── B` B kế thừa A | `I ◄·· C` C cài đặt interface I | `A ◆── B` A chứa B (composition) | `A ──► B` A tham chiếu B

Nhóm 1	**Quản lý Thư viện** Library Manager — Quản lý tài liệu (sách, tạp chí, luận văn), mượn – trả và thống kê lượt mượn
1	Tạo lớp trừu tượng `LibraryItem` gồm các trường private: `id`, `title`, `author`, `year`, `available`; getter/setter có **kiểm tra hợp lệ** (năm 1900 → năm hiện tại, tiêu đề không rỗng). Khai báo 2 phương thức trừu tượng `getLoanDays()` và `getTypeName()` (Encapsulation, Abstraction).
2	Tạo 3 lớp con: `Book` (isbn, pages), `Magazine` (issueNumber), `Thesis` (supervisor, university). Override `getLoanDays()`: Book 14 ngày, Magazine 7 ngày, Thesis 3 ngày; override `toString()` (Inheritance, Polymorphism).
3	Tạo interface `LibraryRepository` với `add`, `remove`, `findById`, `findAll`, `search(keyword)`; cài đặt `InMemoryLibraryRepository` dùng `HashMap<String, LibraryItem>`. Khởi tạo sẵn ≥ 15 tài liệu mẫu.
4	`MainWindow extends BaseWindow`: trong `initUI()` dùng `JSplitPane` — trái là `JTable` (`ItemTableModel extends AbstractTableModel`), phải là panel chi tiết tài liệu đang chọn. Thêm ô tìm kiếm lọc theo tên/tác giả theo thời gian thực (`TableRowSorter` + `RowFilter`).
5	`ItemFormDialog extends JDialog` (modal) để Thêm/Sửa: `JComboBox` chọn loại tài liệu → hiển thị nhóm trường riêng bằng `CardLayout`; kiểm tra ISBN (10 hoặc 13 chữ số), năm xuất bản; báo lỗi rõ ràng.
6	Tạo `LoanRecord` (item, borrowerName, borrowDate, dueDate, returnDate) và `LoanDialog` để Mượn/Trả. Hạn trả = ngày mượn + `item.getLoanDays()`. Tài liệu quá hạn được **tô đỏ** trong bảng bằng `TableCellRenderer` tự viết.
7	`ChartWindow extends JFrame` (DISPOSE_ON_CLOSE) có `JTabbedPane`: (1) PieChart tỷ lệ tài liệu theo loại, (2) BarChart số lượt mượn theo tháng.
BONUS	Lưu/đọc dữ liệu ra file (CSV hoặc `ObjectOutputStream` với `Serializable`); tính tiền phạt trễ hạn theo **đa hình** (`calculateFine(days)` khác nhau theo loại tài liệu).

Khái niệm OOP trọng tâm: Abstraction (`LibraryItem`, `LibraryRepository`), Inheritance (Book/Magazine/Thesis, JDialog, AbstractTableModel), Polymorphism (`getLoanDays()`, cell renderer), Encapsulation (validate trong setter), Composition (`LoanRecord` chứa `LibraryItem`)

Sơ đồ lớp gợi ý (tối thiểu):

```
LibraryItem (abstract) ◄── Book | Magazine | Thesis
LibraryRepository (interface) ◄·· InMemoryLibraryRepository
LoanRecord ──► LibraryItem        ItemTableModel ──► LibraryRepository
BaseWindow ◄── MainWindow ◆── JSplitPane, ItemTableModel, ChartWindow
JDialog ◄── ItemFormDialog | LoanDialog | AboutDialog
```

Kịch bản demo bắt buộc: Thêm 1 cuốn sách mới → mượn sách đó → chỉnh ngày để quá hạn (dòng chuyển đỏ) → tìm kiếm theo tác giả → mở cửa sổ biểu đồ.

Phân bổ 2 giờ trên lớp (gợi ý): Setup 15′ | Model + Repository 30′ | Bảng + tìm kiếm 30′ | Dialog 25′ | Biểu đồ 20′

Nhóm 2	**Quản lý Nhân sự & Bảng lương** HR Payroll System — Quản lý nhân viên nhiều loại hợp đồng, tính lương – thuế và phân tích quỹ lương
1	Tạo lớp trừu tượng `Employee`: `id`, `fullName`, `department`, `baseSalary`, `hireDate` (private + validate); phương thức trừu tượng `calculateSalary()`; phương thức `final int getSeniorityYears()`.
2	Tạo các lớp con `FullTimeEmployee` (phụ cấp, hệ số thưởng), `PartTimeEmployee` (lương giờ, số giờ), `Intern` (trợ cấp cố định) và `Manager extends FullTimeEmployee` (phụ cấp trách nhiệm theo số người quản lý) — kế thừa **2 tầng**, dùng `super.calculateSalary()`.
3	Tạo interface `Taxable { double calculateTax(); }` chỉ cho nhân viên chính thức (FullTime, Manager). Thuế TNCN tính lũy tiến đơn giản theo bảng bậc nhóm tự quy ước (ghi rõ trong báo cáo). `Intern` **không** implement `Taxable`.
4	`MainWindow extends BaseWindow`: `JTabbedPane` gồm tab "Nhân viên" (`JTable` + `JComboBox` lọc phòng ban) và tab "Bảng lương tháng" (lương, thuế, thực lĩnh; dòng tổng cuối bảng).
5	`EmployeeFormDialog extends JDialog`: chọn loại nhân viên → hiển thị trường tương ứng; dùng `JSpinner` cho số giờ; dữ liệu sai thì ném và bắt `InvalidEmployeeDataException` (exception tự định nghĩa).
6	Tạo `PayrollService` tính tổng quỹ lương theo phòng ban (`Map<String, Double>`, có thể dùng Stream `groupingBy`); cập nhật status bar: tổng số nhân viên, tổng quỹ lương.
7	Biểu đồ: BarChart quỹ lương theo phòng ban; PieChart cơ cấu loại nhân viên. Biểu đồ phải **cập nhật lại** khi thêm/sửa/xóa nhân viên.
BONUS	Xuất phiếu lương (payslip) của nhân viên đang chọn ra file `.html`/`.txt`; sắp xếp bảng theo nhiều tiêu chí bằng `Comparator.comparing(...).thenComparing(...)`.

Khái niệm OOP trọng tâm: Abstraction (`Employee`), Inheritance 2 tầng (`Manager → FullTimeEmployee → Employee`), Polymorphism (`calculateSalary()`), Interface (`Taxable`), Encapsulation, Exception tự định nghĩa

Sơ đồ lớp gợi ý (tối thiểu):

```
Employee (abstract) ◄── FullTimeEmployee ◄── Manager
                    ◄── PartTimeEmployee | Intern
Taxable (interface) ◄·· FullTimeEmployee (và Manager kế thừa)
PayrollService ──► List<Employee>     InvalidEmployeeDataException ──► Exception
BaseWindow ◄── MainWindow ◆── JTabbedPane, EmployeeTableModel, PayrollTableModel
```

Kịch bản demo bắt buộc: Thêm 1 Manager và 1 Intern → xem bảng lương (Intern không có thuế) → nhập dữ liệu sai để hiện thông báo lỗi → biểu đồ quỹ lương thay đổi.

Phân bổ 2 giờ trên lớp (gợi ý): Setup 15′ | Phân cấp lớp + tính lương 35′ | Bảng + lọc 25′ | Dialog + exception 25′ | Biểu đồ 20′

Nhóm 3	**Ngân hàng Mini** Mini Banking — Quản lý tài khoản, giao dịch nạp – rút – chuyển khoản và biến động số dư
1	Tạo `Customer` (id, name, phone) và lớp trừu tượng `Account` (accountNumber, owner, `balance` private — **không có** `setBalance()` public); `deposit(amount)` và phương thức trừu tượng `withdraw(amount)`.
2	Tạo `SavingsAccount` (lãi suất, tối đa 3 lần rút/tháng) và `CheckingAccount` (hạn mức thấu chi). Mỗi lớp override `withdraw()` theo quy tắc riêng.
3	Tạo exception tự định nghĩa `InsufficientFundsException`, `InvalidAmountException`; lớp **bất biến (immutable)** `Transaction` (id, type: enum `DEPOSIT/WITHDRAW/TRANSFER`, amount, timestamp, balanceAfter).
4	Tạo `Bank` quản lý `Map<String, Account>`; `transfer(from, to, amount)` phải đảm bảo **toàn vẹn**: nếu một bước lỗi thì không tài khoản nào bị thay đổi.
5	`MainWindow extends BaseWindow`: `JSplitPane` — trái là `JList` tài khoản với `ListCellRenderer` tự viết (icon khác nhau theo loại tài khoản); phải là thông tin tài khoản + `JTable` lịch sử giao dịch.
6	`TransactionDialog extends JDialog` cho Nạp/Rút/Chuyển; định dạng tiền VND; bắt exception và hiển thị bằng `JOptionPane`, không để chương trình văng lỗi.
7	Biểu đồ: TimeSeries/LineChart biến động số dư của tài khoản đang chọn; BarChart tổng nạp – tổng rút theo ngày. Nút "Mô phỏng cuối tháng" cộng lãi cho tài khoản tiết kiệm.
BONUS	`LoginDialog` nhập mã PIN (sai 3 lần → khóa tài khoản); interface `InterestBearing`; xuất sao kê ra CSV.

Khái niệm OOP trọng tâm: Encapsulation chặt (không có setBalance), Abstraction (`Account`), Polymorphism (`withdraw()`), Exception tự định nghĩa, Immutable object (`Transaction`), Composition (`Account` chứa `List<Transaction>`)

Sơ đồ lớp gợi ý (tối thiểu):

```
Account (abstract) ◄── SavingsAccount | CheckingAccount
Account ──► Customer      Account ◆── List<Transaction> (immutable)
Bank ◆── Map<String, Account>     InsufficientFundsException ──► Exception
BaseWindow ◄── MainWindow ◆── JList<Account>, TransactionTableModel, ChartPanel
```

Kịch bản demo bắt buộc: Rút quá số dư (hiện lỗi) → chuyển khoản giữa 2 tài khoản → xem lịch sử và đường biến động số dư → mô phỏng cộng lãi cuối tháng.

Phân bổ 2 giờ trên lớp (gợi ý): Setup 15′ | Model + exception 35′ | Danh sách + lịch sử 25′ | Dialog giao dịch 25′ | Biểu đồ 20′

Nhóm 4	**Đặt phòng Khách sạn** Hotel Booking — Sơ đồ phòng trực quan, đặt phòng theo ngày, kiểm tra trùng lịch và lập hóa đơn
1	Tạo lớp trừu tượng `Room` (roomNumber, floor, status: enum `AVAILABLE/OCCUPIED/MAINTENANCE`) với các phương thức trừu tượng `getPricePerNight()` và `getAmenities()`.
2	Tạo `StandardRoom`, `DeluxeRoom`, `SuiteRoom` (giá và tiện nghi khác nhau) và `Guest` (cccd, name, phone). Khởi tạo khách sạn 4 tầng × 6 phòng.
3	Tạo `Booking` (guest, room, checkIn, checkOut kiểu `LocalDate`) với `nights()`, `totalCost()`. `BookingService` kiểm tra **trùng lịch** của cùng một phòng → ném `RoomNotAvailableException`.
4	`RoomMapPanel extends JPanel`: override `paintComponent()` vẽ sơ đồ phòng theo tầng, tô màu theo trạng thái; dùng `MouseListener` để click chọn phòng và hiển thị chi tiết.
5	`BookingDialog extends JDialog`: chọn ngày bằng `JSpinner` (SpinnerDateModel), tự động tính số đêm và tổng tiền mỗi khi đổi ngày (`ChangeListener`).
6	`JTable` danh sách đặt phòng; chức năng Check-out mở `InvoiceDialog` với các phụ phí dịch vụ (`List<ServiceCharge>`: minibar, giặt ủi…), tổng tiền cuối cùng.
7	Biểu đồ: BarChart công suất phòng (%) theo loại phòng; LineChart doanh thu 14 ngày gần nhất (dữ liệu mô phỏng).
BONUS	interface `PricingPolicy` với `WeekendPricing`, `HolidayPricing` (mẫu Strategy) — đổi chính sách giá trong lúc chạy chương trình.

Khái niệm OOP trọng tâm: Abstraction (`Room`), Inheritance, Polymorphism (`getPricePerNight()`, `paintComponent()`), Encapsulation, Composition (`Booking` – `Guest` – `Room`, `Invoice` chứa `ServiceCharge`), Exception tự định nghĩa

Sơ đồ lớp gợi ý (tối thiểu):

```
Room (abstract) ◄── StandardRoom | DeluxeRoom | SuiteRoom
Booking ──► Guest, Room        Invoice ◆── List<ServiceCharge>
BookingService ──► List<Booking>     PricingPolicy (interface) ◄·· WeekendPricing | HolidayPricing
BaseWindow ◄── MainWindow ◆── RoomMapPanel (extends JPanel), BookingTableModel
```

Kịch bản demo bắt buộc: Click một phòng trống → đặt 3 đêm → thử đặt trùng ngày phòng đó (báo lỗi) → check-out có phụ phí → xem biểu đồ công suất.

Phân bổ 2 giờ trên lớp (gợi ý): Setup 15′ | Model + kiểm tra trùng lịch 30′ | Sơ đồ phòng 30′ | Dialog đặt phòng 25′ | Biểu đồ 20′

Nhóm 5	**Quầy bán hàng Quán cà phê** Coffee Shop POS — Gọi món, áp dụng khuyến mãi, thanh toán và thống kê doanh thu
1	Tạo lớp trừu tượng `MenuItem` (code, name, basePrice, category) với `getPrice()`; lớp con `Drink` (size: enum `S/M/L` có hệ số giá), `Food`, và `Combo` — **Combo cũng là MenuItem và chứa `List<MenuItem>`** (mẫu Composite).
2	Tạo `OrderItem` (menuItem, quantity, note) và `Order` (id, tableNo, `List<OrderItem>`, status: enum, createdAt) với `getSubtotal()`, `getTotal()`.
3	Tạo interface `DiscountStrategy { double apply(Order o); }` với ≥ 3 cài đặt: `NoDiscount`, `PercentDiscount`, `HappyHourDiscount` (14h–16h giảm 20% đồ uống), `MemberDiscount`.
4	`MainWindow extends BaseWindow`: bên trái `MenuPanel` gồm các nút món **sinh tự động** từ danh sách menu (GridLayout, có icon), lọc theo danh mục bằng `JTabbedPane`; bên phải `JTable` giỏ hàng + tổng tiền.
5	`CheckoutDialog extends JDialog`: chọn khuyến mãi, nhập tiền khách đưa → tính tiền thối; in hóa đơn vào `JTextArea`, định dạng tiền Việt Nam bằng `NumberFormat`.
6	`SalesRepository` lưu các đơn đã thanh toán; `ReportWindow extends JFrame` hiển thị đơn hàng theo ngày trong `JTable`.
7	Biểu đồ: BarChart top 5 món bán chạy; LineChart doanh thu theo giờ trong ngày; PieChart doanh thu theo danh mục.
BONUS	Sơ đồ bàn (`TablePanel` vẽ bằng `paintComponent`, màu theo trạng thái bàn); đọc menu từ file CSV.

Khái niệm OOP trọng tâm: Abstraction (`MenuItem`), Inheritance, Polymorphism (`getPrice()` của Drink/Combo; `DiscountStrategy.apply()`), mẫu Composite (Combo) và Strategy (khuyến mãi), Composition (`Order` chứa `OrderItem`), Enum

Sơ đồ lớp gợi ý (tối thiểu):

```
MenuItem (abstract) ◄── Drink | Food | Combo ◆── List<MenuItem>
Order ◆── List<OrderItem> ──► MenuItem
DiscountStrategy (interface) ◄·· NoDiscount | PercentDiscount | HappyHourDiscount | MemberDiscount
BaseWindow ◄── MainWindow ◆── MenuPanel, CartTableModel     JFrame ◄── ReportWindow
```

Kịch bản demo bắt buộc: Gọi 1 đồ uống size L + 1 Combo → áp dụng Happy Hour → thanh toán, tính tiền thối → mở báo cáo và biểu đồ món bán chạy.

Phân bổ 2 giờ trên lớp (gợi ý): Setup 15′ | Model + Strategy 30′ | Menu + giỏ hàng 35′ | Thanh toán 20′ | Biểu đồ 20′

Nhóm 6	**Đăng ký Học phần** Course Registration — Sinh viên đăng ký lớp học phần, kiểm tra ràng buộc và xem thời khóa biểu tuần
1	Tạo lớp trừu tượng `Person` (id, fullName, email) và hai lớp con `Student` (major, cohort, maxCredits) và `Lecturer` (degree, department); override `getRoleDescription()`.
2	Tạo `Course` (code, name, credits, danh sách mã học phần tiên quyết), `ClassSection` (sectionId, course, lecturer, capacity, `List<TimeSlot>`) và `TimeSlot` (thứ, tiết bắt đầu, tiết kết thúc, phòng) có phương thức `overlaps(TimeSlot other)`.
3	`RegistrationService.register(student, section)` kiểm tra: (a) lớp đầy, (b) trùng lịch, (c) vượt số tín chỉ tối đa, (d) chưa học tiên quyết. Mỗi lỗi là **một exception riêng** kế thừa `RegistrationException`.
4	`MainWindow extends BaseWindow`: `LoginDialog` chọn sinh viên khi mở app; `JTable` danh sách lớp học phần (lọc theo khoa/từ khóa); nút Đăng ký / Hủy đăng ký.
5	`TimetablePanel extends JPanel`: override `paintComponent()` vẽ thời khóa biểu tuần (7 cột × 12 tiết), tô màu các học phần đã đăng ký, có tên môn và phòng.
6	Status bar hiển thị tổng tín chỉ đã đăng ký (giới hạn 14–25 TC); chức năng xuất thời khóa biểu ra ảnh PNG (`BufferedImage` + `ImageIO`).
7	Biểu đồ: BarChart tỷ lệ lấp đầy các lớp (số đã đăng ký / sĩ số); PieChart phân bố tín chỉ theo khối kiến thức.
BONUS	Chế độ Giảng viên — đăng nhập bằng `Lecturer` sẽ thấy giao diện khác (danh sách sinh viên của lớp mình dạy), thực hiện bằng **đa hình** thay vì if-else theo vai trò.
Khái niệm OOP trọng tâm: Abstraction & Inheritance (`Person`), Polymorphism (`getRoleDescription()`, giao diện theo vai trò), Encapsulation, Composition (`ClassSection` chứa `TimeSlot`), phân cấp Exception (`RegistrationException`)	

Sơ đồ lớp gợi ý (tối thiểu):
`Person (abstract) ◄── Student | Lecturer`
`ClassSection ──► Course, Lecturer  ClassSection ◆── List<TimeSlot>`
`Enrollment (Student ⇔ ClassSection)  RegistrationException ◄── SectionFullException |`
`ScheduleConflictException | CreditLimitException | PrerequisiteException`
`BaseWindow ◄── MainWindow ◆── SectionTableModel, TimetablePanel (extends JPanel)`	

Kịch bản demo bắt buộc: Đăng nhập 1 sinh viên → đăng ký 3 lớp → thử đăng ký lớp trùng lịch và lớp thiếu tiên quyết (2 thông báo khác nhau) → xem TKB → xuất PNG.	
Phân bổ 2 giờ trên lớp (gợi ý): Setup 15' | Model + kiểm tra ràng buộc 35' | Bảng lớp HP 20' | Vẽ TKB 30' | Biểu đồ 20'	

Nhóm 7	**Trình soạn thảo Văn bản Mini** Mini Text Editor — Soạn thảo nhiều tab, mở/lưu file, tìm – thay thế, hoàn tác và thống kê văn bản
1	Tạo `DocumentModel` (filePath, content, `modified`, encoding) với đóng gói đầy đủ: khi nội dung thay đổi thì cờ `modified` bật và tiêu đề tab có dấu `*`.
2	`EditorTab extends JPanel` chứa `JTextArea` trong `JScrollPane` và `LineNumberPanel extends JPanel` (override `paintComponent()` vẽ số dòng, gắn bằng `setRowHeaderView`).
3	`MainWindow extends BaseWindow`: `JTabbedPane` mở nhiều tài liệu; menu File: New / Open / Save / Save As / Close Tab (`JFileChooser`, đọc ghi UTF-8 bằng `Files.readString` / `Files.writeString`).
4	Khi đóng tab hoặc thoát chương trình mà còn tài liệu chưa lưu → hỏi Lưu / Không lưu / Hủy. Gợi ý: `setDefaultCloseOperation(DO_NOTHING_ON_CLOSE)` + `WindowAdapter.windowClosing()`.
5	`FindReplaceDialog extends JDialog` (**non-modal**): Tìm tiếp, Thay thế, Thay tất cả, tùy chọn phân biệt hoa/thường; tô vàng kết quả bằng `Highlighter`.
6	Undo/Redo bằng `UndoManager`; phím tắt Ctrl+N/O/S/F/Z/Y bằng `KeyStroke`; status bar hiển thị Dòng:Cột, số từ, số ký tự (`CaretListener`, `DocumentListener`).
7	Tạo interface `TextAnalyzer` với `WordFrequencyAnalyzer` và `CharCategoryAnalyzer`; `StatisticsWindow` gồm BarChart top 10 từ xuất hiện nhiều nhất và PieChart tỷ lệ chữ cái / chữ số / khoảng trắng / ký tự khác.
BONUS	Tô màu từ khóa Java (`StyledDocument`); danh sách Recent Files lưu bằng `Properties`; chuyển theme sáng/tối.

Khái niệm OOP trọng tâm: Encapsulation (`DocumentModel`), Inheritance (EditorTab, LineNumberPanel, JDialog), Polymorphism (`paintComponent()`, `TextAnalyzer.analyze()`), Composition (MainWindow chứa nhiều EditorTab), xử lý I/O và Exception

Sơ đồ lớp gợi ý (tối thiểu):

```
DocumentModel     EditorTab (extends JPanel) ◆── JTextArea, LineNumberPanel, DocumentModel
TextAnalyzer (interface) ◂·· WordFrequencyAnalyzer | CharCategoryAnalyzer
BaseWindow ◂── MainWindow ◆── JTabbedPane<EditorTab>, UndoManager
JDialog ◂── FindReplaceDialog (non-modal)     JFrame ◂── StatisticsWindow
```

Kịch bản demo bắt buộc: Mở 1 file .txt tiếng Việt → sửa (tab có *) → Tìm & Thay tất cả → Undo → đóng app khi chưa lưu (hiện hộp hỏi) → mở thống kê.

Phân bổ 2 giờ trên lớp (gợi ý): Setup 15′ | Tab + mở/lưu file 35′ | Find/Replace 25′ | Undo + status 20′ | Thống kê + biểu đồ 25′

Nhóm 8	**Mô phỏng Nút giao thông** Traffic Intersection Simulator — Mô phỏng hoạt hình ngã tư có đèn tín hiệu và nhiều loại phương tiện
1	Tạo enum `LightState { GREEN, YELLOW, RED }` có `next()` và thời lượng mặc định; lớp `TrafficLight` (state, remainingSeconds, direction) hoạt động như **máy trạng thái**.
2	Tạo lớp trừu tượng `Vehicle` (x, y, speed, direction, color) với `abstract void draw(Graphics2D g)`, `move()`, `shouldStop(TrafficLight l)`; lớp con `Car`, `Bus` (dài, chậm), `Motorbike` (nhỏ, nhanh), `Ambulance` (override `shouldStop()` → không dừng đèn đỏ).
3	`IntersectionPanel extends JPanel`: override `paintComponent()` vẽ ngã tư, vạch dừng, đèn tín hiệu và toàn bộ xe (Graphics2D, bật antialiasing).
4	Bộ máy mô phỏng dùng `javax.swing.Timer` (30–50 ms/khung hình): `VehicleFactory` sinh xe ngẫu nhiên theo tỷ lệ; xe dừng ở vạch khi đèn đỏ, giữ khoảng cách với xe phía trước; xóa xe khi ra khỏi màn hình.
5	Toolbar: Start / Pause / Reset / x2 tốc độ; `JSlider` chỉnh mật độ xe; status bar: thời gian mô phỏng, số xe đang có, số xe đã qua.
6	`SettingsDialog extends JDialog`: thời lượng xanh/vàng/đỏ cho từng hướng (`JSpinner`), tỷ lệ từng loại xe.
7	Biểu đồ: XYLineChart cập nhật theo thời gian thực số xe đang chờ ở mỗi hướng; BarChart số xe đã qua ngã tư theo loại.
BONUS	interface `LightController` với `FixedTimeController` và `AdaptiveController` (ưu tiên hướng đông xe); so sánh thời gian chờ trung bình của 2 chiến lược trên biểu đồ.
Khái niệm OOP trọng tâm: Abstraction (`Vehicle`), Polymorphism (`draw()`, `move()`, `shouldStop()`), Inheritance, Enum có hành vi (máy trạng thái), mẫu Factory, Composition (Intersection chứa đèn và danh sách xe)	

Sơ đồ lớp gợi ý (tối thiểu):
`Vehicle (abstract) ◄── Car | Bus | Motorbike | Ambulance`
`LightState (enum)  TrafficLight ──► LightState  VehicleFactory ──► Vehicle`
`Simulation ◆── List<Vehicle>, TrafficLight[4]  LightController (interface) ◄·· Fixed | Adaptive`
`BaseWindow ◄── MainWindow ◆── IntersectionPanel (extends JPanel), ChartPanel`	

Kịch bản demo bắt buộc: Start → xe dừng đúng vạch khi đèn đỏ → xe cứu thương vượt đèn đỏ → đổi thời lượng đèn trong Settings → quan sát biểu đồ xe chờ thay đổi.	
Phân bổ 2 giờ trên lớp (gợi ý): Setup 15′ | Đèn + Vehicle 30′ | Vẽ ngã tư 30′ | Vòng lặp mô phỏng 30′ | Biểu đồ 15′	

Nhóm 9	**Giám sát Ao nuôi tôm IoT** Smart Aquaculture Monitor — Dashboard giám sát cảm biến chất lượng nước theo thời gian thực và cảnh báo vượt ngưỡng
1	Tạo lớp trừu tượng `Sensor` (sensorId, name, unit, minSafe, maxSafe) với `abstract double read()`; lớp con `TemperatureSensor` (26–32 °C), `PHSensor` (7,5–8,5), `DissolvedOxygenSensor` (DO ≥ 4 mg/L), `SalinitySensor` (10–25 ‰). Dữ liệu mô phỏng dao động ngẫu nhiên (`Random.nextGaussian()`), thỉnh thoảng vượt ngưỡng.
2	Tạo `Pond` (pondId, name, area, `List<Sensor>`), `Farm` chứa nhiều `Pond`, và lớp bất biến `Reading` (sensorId, value, timestamp).
3	Áp dụng mẫu **Observer**: interface `SensorListener { void onReading(Reading r); void onAlert(Alert a); }`; `SensorHub` phát sự kiện, các panel đăng ký lắng nghe.
4	`MainWindow extends BaseWindow`: `JComboBox` chọn ao; dashboard dạng lưới gồm các `GaugePanel extends JPanel` (override `paintComponent()` vẽ đồng hồ đo/thanh màu xanh – vàng – đỏ).
5	`javax.swing.Timer` mỗi 1 giây đọc cảm biến → cập nhật gauge, `JTable` nhật ký (chỉ giữ 200 dòng gần nhất); khi vượt ngưỡng: đổi màu gauge, ghi vào bảng cảnh báo, status bar báo đỏ.
6	`ThresholdDialog extends JDialog`: chỉnh ngưỡng an toàn từng cảm biến; lưu/đọc bằng `java.util.Properties`.
7	Biểu đồ: TimeSeriesChart thời gian thực (60 giây gần nhất) cho 4 chỉ số; BarChart số lần cảnh báo theo từng cảm biến.
BONUS	interface `Controllable` cho thiết bị chấp hành (`Aerator` – máy sục khí) tự bật khi DO dưới ngưỡng và tắt khi ổn định; xuất dữ liệu ra CSV.

Khái niệm OOP trọng tâm: Abstraction (`Sensor`), Polymorphism (`read()`, `paintComponent()`), mẫu Observer (`SensorListener`), Immutable (`Reading`), Composition (`Farm` → `Pond` → `Sensor`), Interface (`Controllable`)

Sơ đồ lớp gợi ý (tối thiểu):

```
Sensor (abstract) ◄── TemperatureSensor | PHSensor | DissolvedOxygenSensor | SalinitySensor
Farm ◆── List<Pond> ◆── List<Sensor>        Reading (immutable), Alert
SensorListener (interface) ◄·· GaugePanel | LogTableModel | ChartController ; SensorHub ──►
List<SensorListener>
BaseWindow ◄── MainWindow ◆── GaugePanel[4], JTable, ChartPanel
```

Kịch bản demo bắt buộc: Chạy dashboard → chờ DO tụt dưới ngưỡng (gauge đỏ, có cảnh báo) → hạ/nâng ngưỡng trong ThresholdDialog → biểu đồ realtime cập nhật liên tục.

Phân bổ 2 giờ trên lớp (gợi ý): Setup 15′ | Sensor + Observer 30′ | Gauge vẽ tay 30′ | Timer + nhật ký 20′ | Biểu đồ realtime 25′

Nhóm 10	**Trò chơi Rắn săn mồi** Snake Game — Game 2D có nhiều màn chơi, nhiều loại mồi và bảng xếp hạng điểm
1	Tạo lớp trừu tượng `GameObject` (position, color) với `abstract void draw(Graphics2D g, int cellSize)`; lớp con `Snake` (`LinkedList<Point>` thân rắn, hướng), `Food`, `BonusFood` (biến mất sau 5 giây, điểm ×3), `Obstacle`.
2	Tạo enum `Direction { UP, DOWN, LEFT, RIGHT }` với `isOpposite(Direction d)` — không cho rắn quay đầu 180°; enum `GameState { READY, RUNNING, PAUSED, GAME_OVER }`.
3	`GameBoardPanel extends JPanel`: override `paintComponent()` vẽ lưới, rắn, mồi, vật cản, điểm; điều khiển bằng **Key Bindings** (`InputMap`/`ActionMap`) thay vì `KeyListener`.
4	`GameEngine` dùng `javax.swing.Timer`: mỗi tick → di chuyển → kiểm tra va chạm (tường, thân, vật cản) → ăn mồi → tăng điểm và tăng tốc (`timer.setDelay`).
5	Tạo interface `LevelConfig` với `Level1` (không vật cản), `Level2` (tường viền), `Level3` (mê cung) — mỗi level tự tạo vật cản qua `createObstacles()` (đa hình).
6	`GameOverDialog extends JDialog` nhập tên → tạo `ScoreRecord` (name, score, level, date); `HighScoreRepository` lưu file; `HighScoreDialog` hiển thị top 10 trong `JTable`.
7	Biểu đồ: LineChart điểm số các lượt chơi gần nhất; BarChart điểm trung bình theo level.
BONUS	Âm thanh khi ăn mồi (`javax.sound.sampled`); `PoisonFood` làm rắn ngắn lại; chế độ 2 người chơi.

Khái niệm OOP trọng tâm: Abstraction (`GameObject`), Inheritance (Food → BonusFood), Polymorphism (`draw()`, `LevelConfig.createObstacles()`), Enum có hành vi, Encapsulation (Snake tự quản lý thân), Composition (GameEngine chứa Snake, Food, Level)

Sơ đồ lớp gợi ý (tối thiểu):

```
GameObject (abstract) ◄── Snake | Food ◄── BonusFood | Obstacle
Direction (enum), GameState (enum)    LevelConfig (interface) ◄·· Level1 | Level2 | Level3
GameEngine ◆── Snake, List<Food>, List<Obstacle>, LevelConfig, javax.swing.Timer
BaseWindow ◄── MainWindow ◆── GameBoardPanel (extends JPanel)    HighScoreRepository ──► ScoreRecord
```

Kịch bản demo bắt buộc: Chơi Level 1 ăn vài mồi (tốc độ tăng) → Pause/Resume → chuyển Level 3 → game over, nhập tên → bảng xếp hạng và biểu đồ điểm.

Phân bổ 2 giờ trên lớp (gợi ý): Setup 15′ | GameObject + Snake 30′ | Vẽ bàn chơi + điều khiển 30′ | Engine + va chạm 25′ | Điểm + biểu đồ 20′

7. Hướng dẫn thuyết trình (10 phút + 5 phút phản biện)

#	Phần trình bày	Nội dung cần có	Thời gian
1	**Giới thiệu**	Tên đề tài, thành viên & vai trò, mục tiêu ứng dụng (một câu: ứng dụng giải quyết vấn đề gì).	1’
2	**Demo trực tiếp**	Chạy ứng dụng **theo đúng kịch bản demo** trong thẻ đề: menu, toolbar, dialog, biểu đồ, một tình huống lỗi được xử lý.	3’
3	**Thiết kế**	Sơ đồ lớp UML: chỉ ra lớp trừu tượng, interface, quan hệ kế thừa/kết hợp. Giải thích **một quyết định thiết kế** (vì sao chọn như vậy).	1,5’
4	**OOP trong code**	Mở Eclipse: BaseWindow → MainWindow; một ví dụ đa hình thực sự (gọi qua kiểu cha); đóng gói có kiểm tra; interface tự định nghĩa.	3’
5	**Khó khăn & bài học**	Một lỗi/khó khăn kỹ thuật cụ thể, cách nhóm tìm ra nguyên nhân và cách khắc phục.	1’
6	**Kết luận**	Mức độ hoàn thành (bảng ✓/✗ 8 bước), hướng phát triển.	0,5’
7	**Phản biện**	Giảng viên và các nhóm khác đặt câu hỏi. GV hỏi **đích danh** từng thành viên, có thể yêu cầu sửa code trực tiếp trong 3–5 phút.	5’

Dàn ý slide gợi ý (10–12 slide)

Slide	Nội dung
1	Tiêu đề: tên đề tài, số nhóm, danh sách thành viên + MSSV
2	Bài toán & chức năng chính (sơ đồ use case hoặc danh sách chức năng)
3	Phân công công việc & tỷ lệ đóng góp (lấy từ Git + bảng tự đánh giá)
4–5	Sơ đồ lớp UML (toàn cảnh + phóng to phần phân cấp chính)
6	Bảng ánh xạ "Nguyên lý OOP → lớp/phương thức cụ thể trong code"
7	Ảnh chụp màn hình các cửa sổ chính (dự phòng khi demo gặp sự cố)
8	Biểu đồ JFreeChart: dữ liệu gì, trả lời câu hỏi nghiệp vụ nào
9	Khó khăn & giải pháp
10	Bảng mức độ hoàn thành 8 bước + BONUS
11	Kết luận & hướng phát triển; Q&A

Mẹo thuyết trình

- Chạy thử ứng dụng trên máy trình chiếu **trước** buổi bảo vệ
- Tăng cỡ chữ editor Eclipse lên ≥ 16 để cả lớp đọc được
- Mỗi người trình bày phần mình viết code
- Chuẩn bị video demo dự phòng (≤ 3 phút)
- Nói **vì sao** dùng OOP, không chỉ đọc định nghĩa

Những lỗi thường gặp

- Đọc slide, không mở code
- Demo bằng dữ liệu "đẹp", không thử trường hợp lỗi
- Chỉ 1–2 người nói, người khác đứng im
- Không trả lời được code của chính mình
- Biểu đồ không có tiêu đề, nhãn trục, đơn vị

8. Hướng dẫn viết báo cáo & nộp sản phẩm

8.1 Hình thức

- Khổ A4, font Times New Roman 13, giãn dòng 1,3; lề trái 3 cm, các lề khác 2 cm; có đánh số trang.
- Độ dài: **15–25 trang** (không tính phụ lục). Nộp bản **PDF**.
- Mã nguồn trích dẫn trong báo cáo: chỉ trích **đoạn quan trọng** (≤ 20 dòng/đoạn), font Consolas 10, có chú thích giải thích.
- Hình ảnh, bảng phải có số thứ tự và tên (Hình 2.1, Bảng 3.2…).

8.2 Cấu trúc báo cáo

Phần	Nội dung chi tiết	Trang
Bìa	Học viện, khoa, môn học (INT1332 – Lập trình hướng đối tượng), tên đề tài, nhóm, thành viên + MSSV, GV hướng dẫn, ngày nộp.	1
Bảng phân công & đóng góp	Bảng: Họ tên – MSSV – Vai trò – Các lớp/chức năng đã thực hiện – % đóng góp (tổng 100%) – Chữ ký xác nhận của cả nhóm.	1
Chương 1. Giới thiệu	Bối cảnh bài toán, mục tiêu, phạm vi, danh sách chức năng (đánh dấu bắt buộc/BONUS).	1–2
Chương 2. Phân tích & thiết kế	Sơ đồ use case (tùy chọn); **sơ đồ lớp UML đầy đủ** (thuộc tính, phương thức, quan hệ); mô tả vai trò từng lớp; giải thích các quyết định thiết kế (vì sao lớp trừu tượng, vì sao interface, vì sao composition).	4–6
Chương 3. Cài đặt	Bảng ánh xạ nguyên lý OOP → lớp/phương thức; trích đoạn code minh họa từng nguyên lý kèm giải thích; xử lý sự kiện; tích hợp JFreeChart; xử lý ngoại lệ.	4–6
Chương 4. Kết quả	Ảnh chụp màn hình từng cửa sổ/dialog/biểu đồ có chú thích; bảng mức độ hoàn thành 8 bước.	3–5
Chương 5. Kiểm thử	Bảng ≥ 10 test case: Mã – Mô tả – Dữ liệu vào – Kết quả mong đợi – Kết quả thực tế – Đạt/Không đạt. Bắt buộc có test dữ liệu sai/biên.	1–2
Chương 6. Kết luận	Kết quả đạt được, hạn chế, khó khăn và bài học, hướng phát triển.	1
Tài liệu tham khảo	Trình bày theo chuẩn IEEE hoặc APA 7.	—
Phụ lục A. Nhật ký sử dụng AI	Công cụ AI đã dùng, mục đích, các prompt chính, phần code chịu ảnh hưởng, nhóm đã kiểm chứng/chỉnh sửa gì. **Bắt buộc** (ghi "Không sử dụng" nếu không dùng).	—
Phụ lục B. Lịch sử Git	Ảnh chụp/trích xuất `git log --oneline --graph` hoặc biểu đồ contributors.	—

8.3 Sản phẩm nộp

Nộp **một file nén** tên `OOP_NhomXX_TenDeTai.zip` lên hệ thống LMS/Google Classroom trước **23:59 ngày …/…/2026** (trước buổi thuyết trình ít nhất 1 ngày). Nộp trễ trừ 10%/ngày.

```
OOP_NhomXX_TenDeTai.zip
├── source/              // Toàn bộ project Eclipse (KHÔNG kèm thư mục bin/, target/)
├── app/OOP_NhomXX.jar   // File JAR chạy được: java -jar OOP_NhomXX.jar
├── report/BaoCao_NhomXX.pdf
├── slides/Slide_NhomXX.pptx (hoặc .pdf)
```

```
├── docs/                // Sơ đồ UML (ảnh + file nguồn .drawio/.puml)
└── README.md            // Link Git, link video demo (nếu có), hướng dẫn chạy
```

8.4 Quy định sử dụng AI & liêm chính học thuật

- Được phép dùng công cụ AI để tra cú pháp, gợi ý, sửa lỗi — nhưng **mọi thành viên phải hiểu toàn bộ code** của nhóm.
- Phải khai báo trung thực trong Phụ lục A. Không khai báo mà bị phát hiện → trừ 20% điểm nhóm.
- Trong phản biện, sinh viên **không giải thích được đoạn code mình đứng tên** → 0 điểm cho câu hỏi đó, giảng viên có thể xem xét lại tỷ lệ đóng góp.
- Sao chép code giữa các nhóm hoặc từ bài các khóa trước → 0 điểm cả nhóm.
- Commit thường xuyên với thông điệp có nghĩa (vd. `feat: add LoanDialog with due date check`). Giảng viên sẽ xem lịch sử Git.

9. Thang điểm (100 điểm + 5 điểm BONUS)

Phần	Tiêu chí	Mô tả	Điểm
A. Sản phẩm *(điểm nhóm)*	Thiết kế & cài đặt OOP	Phân cấp lớp có nghĩa; đủ 4 nguyên lý + interface tự định nghĩa; đa hình thực sự (không lạm dụng instanceof).	20
	Giao diện đầy đủ	Đủ thành phần mục 4: MainWindow, ≥ 2 JDialog, menu, toolbar icon, status bar, icon cửa sổ.	12
	JFreeChart	≥ 2 biểu đồ khác loại, dữ liệu đúng và cập nhật động, có tiêu đề/nhãn trục/đơn vị.	8
	Chức năng	Bước 1–7 hoạt động đúng; xử lý dữ liệu sai; không văng exception.	10
	Chất lượng code	Đặt tên đúng quy ước Java, chia package hợp lý, không trùng lặp code, có chú thích ở phương thức quan trọng, lịch sử Git rõ ràng.	5
B. Báo cáo *(điểm nhóm)*	Báo cáo kỹ thuật	Đúng cấu trúc mục 8.2; UML đầy đủ, giải thích quyết định thiết kế; test case; phụ lục AI & Git.	15
C. Thuyết trình *(điểm nhóm)*	Trình bày & demo	Đúng thời gian, demo theo kịch bản, mọi thành viên đều trình bày, slide rõ ràng.	10
D. Phản biện *(điểm cá nhân)*	Trả lời câu hỏi	Trả lời đúng, giải thích được lý do, chỉ ra được đúng vị trí trong code; hoàn thành yêu cầu sửa code trực tiếp (nếu có).	20
BONUS	Bước 8 + mở rộng	Hoàn thành bước 8 hoặc tính năng mở rộng có giá trị (tối đa +5, tổng không vượt 100).	+5

Điểm cá nhân = A + B + C (điểm chung của nhóm, tối đa 80) + D (điểm phản biện của riêng từng sinh viên, tối đa 20) + BONUS. Giảng viên có thể điều chỉnh điểm chung của cá nhân nếu bảng đóng góp hoặc lịch sử Git cho thấy chênh lệch lớn (ví dụ đóng góp < 10%).

10. Đoạn mã tham khảo

10.1 Nạp icon từ resources

```
ImageIcon icon = new ImageIcon(getClass().getResource("/resources/icons/app.png"));
setIconImage(icon.getImage());   // getResource trả về null nếu sai đường dẫn → kiểm tra trước
```

10.2 Biểu đồ cột (BarChart)

```
DefaultCategoryDataset dataset = new DefaultCategoryDataset();
dataset.addValue(85.0, "Điểm", "Toán");
dataset.addValue(72.0, "Điểm", "Tiếng Anh");
JFreeChart chart = ChartFactory.createBarChart(
        "Điểm trung bình theo môn", "Môn học", "Điểm", dataset);
ChartPanel chartPanel = new ChartPanel(chart);
mainPanel.add(chartPanel, BorderLayout.CENTER);
// Cập nhật: dataset.setValue(...) → biểu đồ tự vẽ lại (cơ chế Observer của JFreeChart)
```

10.3 Biểu đồ tròn (PieChart) – JFreeChart 1.5.x

```
DefaultPieDataset<String> pie = new DefaultPieDataset<>();
pie.setValue("Sách", 12);
pie.setValue("Tạp chí", 5);
JFreeChart chart = ChartFactory.createPieChart("Tỷ lệ theo loại", pie, true, true, false);
```

10.4 Biểu đồ thời gian thực (TimeSeries)

```
TimeSeries series = new TimeSeries("Nhiệt độ");
series.setMaximumItemAge(60);                        // chỉ giữ 60 giây gần nhất
TimeSeriesCollection ds = new TimeSeriesCollection(series);
JFreeChart chart = ChartFactory.createTimeSeriesChart(
        "Nhiệt độ ao 1", "Thời gian", "°C", ds);
// Trong javax.swing.Timer (mỗi 1 giây):
series.addOrUpdate(new Second(), sensor.read());
```

10.5 Swing Timer cập nhật định kỳ

```
Timer timer = new Timer(1000, e -> {      // javax.swing.Timer – callback chạy trên EDT
    refreshData();
    setStatus("Cập nhật lúc " + LocalTime.now().withNano(0));
});
timer.start();
```

10.6 Mở dialog con và nhận kết quả

```
ItemFormDialog dialog = new ItemFormDialog(this);   // truyền JFrame cha
dialog.setModal(true);
dialog.setVisible(true);                            // chặn tại đây đến khi dialog đóng
if (dialog.isConfirmed()) {
    LibraryItem item = dialog.getResult();
    repo.add(item);
    tableModel.fireTableDataChanged();
}
```

10.7 Font tiếng Việt cho JFreeChart

```
StandardChartTheme theme = (StandardChartTheme) StandardChartTheme.createJFreeTheme();
theme.setExtraLargeFont(new Font("Segoe UI", Font.BOLD, 18));   // tiêu đề
theme.setLargeFont(new Font("Segoe UI", Font.BOLD, 14));        // nhãn trục
theme.setRegularFont(new Font("Segoe UI", Font.PLAIN, 12));     // chú giải, giá trị
ChartFactory.setChartTheme(theme);       // gọi MỘT lần trong main(), trước khi tạo biểu đồ
```

10.8 TableModel tự viết

```
public class ItemTableModel extends AbstractTableModel {
    private final String[] cols = {"Mã", "Tiêu đề", "Tác giả", "Loại", "Trạng thái"};
    private List<LibraryItem> data;
    public ItemTableModel(List<LibraryItem> data) { this.data = data; }
```

```
    @Override public int getRowCount()            { return data.size(); }
    @Override public int getColumnCount()         { return cols.length; }
    @Override public String getColumnName(int c)  { return cols[c]; }
    @Override public Object getValueAt(int r, int c) {
        LibraryItem it = data.get(r);
        switch (c) {
            case 0: return it.getId();
            case 1: return it.getTitle();
            case 2: return it.getAuthor();
            case 3: return it.getTypeName();          // đa hình
            default: return it.isAvailable() ? "Có sẵn" : "Đang mượn";
        }
    }
    public LibraryItem getItemAt(int modelRow) { return data.get(modelRow); }
}
// Khi bảng có sắp xếp/lọc: int modelRow = table.convertRowIndexToModel(table.getSelectedRow());
```

10.9 Lọc JTable theo ô tìm kiếm

```
TableRowSorter<ItemTableModel> sorter = new TableRowSorter<>(model);
table.setRowSorter(sorter);
searchField.getDocument().addDocumentListener(new DocumentListener() {
    private void filter() {
        String kw = searchField.getText().trim();
        sorter.setRowFilter(kw.isEmpty() ? null : RowFilter.regexFilter("(?i)" + Pattern.quote(kw),
1, 2));
    }
    public void insertUpdate(DocumentEvent e)  { filter(); }
    public void removeUpdate(DocumentEvent e)  { filter(); }
    public void changedUpdate(DocumentEvent e) { filter(); }
});
```

10.10 Exception tự định nghĩa

```
public class InsufficientFundsException extends Exception {   // checked exception
    private final double shortage;
    public InsufficientFundsException(String msg, double shortage) {
        super(msg);
        this.shortage = shortage;
    }
    public double getShortage() { return shortage; }
}
// Sử dụng trong GUI:
try {
    account.withdraw(amount);
} catch (InsufficientFundsException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage(), "Không đủ số dư",
JOptionPane.WARNING_MESSAGE);
}
```

10.11 Tác vụ nền với SwingWorker

```
new SwingWorker<List<Student>, Integer>() {
    @Override protected List<Student> doInBackground() throws Exception {
        return CsvReader.read(file, p -> publish(p));    // chạy ở luồng nền
    }
    @Override protected void process(List<Integer> chunks) {
        progressBar.setValue(chunks.get(chunks.size() - 1));   // chạy trên EDT
    }
    @Override protected void done() {
        try { tableModel.setData(get()); } catch (Exception ex) { showError(ex); }
    }
}.execute();
```

10.12 Key Bindings & lưu cấu hình

```
// Key Binding (không phụ thuộc focus như KeyListener)
InputMap im = panel.getInputMap(JComponent.WHEN_IN_FOCUSED_WINDOW);
im.put(KeyStroke.getKeyStroke("UP"), "moveUp");
panel.getActionMap().put("moveUp", new AbstractAction() {
```

```
    @Override public void actionPerformed(ActionEvent e) { engine.changeDirection(Direction.UP); }
});

// Lưu cấu hình bằng Properties
Properties p = new Properties();
p.setProperty("home.city", "Ho Chi Minh");
try (Writer w = Files.newBufferedWriter(Path.of("app.properties"), StandardCharsets.UTF_8)) {
    p.store(w, "Settings");
}
```

11. Checklist trước khi nộp & thuyết trình

Code

- ☐ MainWindow kế thừa `BaseWindow`, cài đặt `initUI()`
- ☐ Phân cấp lớp nghiệp vụ ≥ 3 lớp con
- ☐ ≥ 1 interface và ≥ 1 exception tự định nghĩa
- ☐ Đa hình gọi qua kiểu cha, không lạm dụng instanceof
- ☐ ≥ 2 JDialog hoạt động đúng (có About)
- ☐ Menu File/View/Help, toolbar icon, status bar
- ☐ ≥ 2 biểu đồ khác loại, có tiêu đề & nhãn trục
- ☐ Nhập dữ liệu sai không làm văng chương trình
- ☐ Tiếng Việt hiển thị đúng ở mọi nơi
- ☐ File JAR chạy được trên máy khác

Báo cáo & thuyết trình

- ☐ Báo cáo PDF đúng cấu trúc, có UML đầy đủ
- ☐ Bảng phân công có chữ ký cả nhóm
- ☐ Phụ lục nhật ký AI + lịch sử Git
- ☐ ≥ 10 test case, có trường hợp lỗi/biên
- ☐ Slide 10–12 trang, có ảnh màn hình dự phòng
- ☐ Đã tập kịch bản demo, tổng thời gian ≤ 10 phút
- ☐ Mỗi thành viên có phần nói riêng
- ☐ Mỗi thành viên giải thích được **mọi** lớp của nhóm
- ☐ Cỡ chữ Eclipse ≥ 16 khi trình chiếu
- ☐ File zip đặt tên đúng quy định, nộp đúng hạn

Chúc các nhóm làm việc hiệu quả!

Hiểu rõ code mình viết và làm việc nhóm tốt quan trọng hơn một giao diện hoàn hảo đến từng điểm ảnh.